

AI-Powered Delay Prediction & Supply Chain Risk Monitor

Assignment

Objective

When a logistics operations team wants to understand the health of their supply chain, they currently rely on manual spreadsheet reviews, reactive escalations, and gut-feel decisions. A shipment delay is discovered after it happens. A problematic carrier is flagged only after multiple complaints. A high-risk route is identified only after a loss occurs.

Your objective is to build a **Supply Chain Intelligence Platform** — an end-to-end system that ingests synthetic shipment data, runs machine learning models to predict delays, detect carrier anomalies, and score route risk, exposes those results through a backend API, and presents everything through an operational web dashboard that a logistics manager can use without any technical knowledge.

By the end of this assignment you will have connected four real components: a synthetic data generator, a trained ML pipeline, a backend REST API with a database, and a React frontend — all running together inside Docker.

PROBLEM STATEMENT

You are a Technical Analyst at FreightIQ, a global logistics company managing shipments across 40+ countries. The Head of Operations has raised three specific problems your platform must solve:

- The team cannot predict which in-transit shipments are at risk of delay before the delay actually happens
- The team has no way to identify which carriers are behaving abnormally compared to the rest of the fleet
- The team has no visibility into which routes are historically dangerous to operate on

Your manager has asked you to build a system that:

- Generates a realistic synthetic shipment dataset large enough to train meaningful models

- Trains ML models to predict delay, estimate delay severity, detect carrier anomalies, and score route risk
- Stores all results in a database and exposes them through a clean REST API
- Presents the data through an operational dashboard a non-technical manager can use

You will build the data, the models, the backend, and the frontend. All four must work together. This is not a collection of independent scripts — it is one connected system.

TASKS TO BE COMPLETED

Task 1 — Generate the Synthetic Dataset

There is no provided dataset. You must generate one using Python. Use libraries such as Faker, NumPy, and Pandas to construct a realistic synthetic shipment dataset with a minimum of 500 records.

Each record must contain the following fields exactly:

- shipment_id — unique identifier (e.g. SHP-0001)
- origin_country — country of shipment origin
- destination_country — country of delivery
- carrier_name — name of the logistics carrier
- shipment_mode — one of: Air, Sea, Road, Rail
- cargo_category — one of: Electronics, Pharmaceuticals, Perishables, Industrial, Consumer Goods
- weight_kg — numeric weight of the cargo
- volume_cbm — cubic meter volume of the cargo
- declared_value_usd — declared monetary value of the shipment
- scheduled_departure_date — planned departure date

- `actual_departure_date` — real departure date (may differ from scheduled)
- `scheduled_delivery_date` — planned delivery date
- `actual_delivery_date` — real delivery date (null for in-transit shipments)
- `customs_cleared` — boolean (true or false)
- `delay_days` — integer: positive means delayed, 0 means on time, negative means early
- `is_delayed` — binary label: 1 if `delay_days` is greater than 0, else 0
- `status` — one of: Delivered, In Transit, Delayed, Held at Customs, Lost

The dataset must reflect realistic distributions — not all shipments should be delayed, not all carriers should be bad, and delay rates must vary meaningfully by route, carrier, and cargo type. A dataset where 90% of shipments are delayed or all carriers behave identically will be penalized. Save the dataset as `shipment_data.csv` and include it in your repository.

Write a short comment block at the top of your data generation script explaining the distributions you chose and why they reflect real logistics behavior.

Task 2 — Build the ML Pipeline

Write a Python module named `pipeline.py` that reads `shipment_data.csv` and trains the following four models. Each model must be trained, evaluated, saved, and documented.

Model 1 — Delay Classifier

- Train a classification model to predict `is_delayed` (0 or 1) using features available before a shipment departs: `origin_country`, `destination_country`, `carrier_name`, `shipment_mode`, `cargo_category`, `weight_kg`, `volume_cbm`, `declared_value_usd`, and `customs_cleared`.
- Split the data 80/20 for training and testing.

- Use Random Forest, XGBoost, or Gradient Boosting — your choice, but justify it in a comment. Evaluate using precision, recall, and F1 score.
- Print the full classification report. The model must achieve a minimum F1 score of 0.70 on the test set.
- Save the trained model as `models/delay_classifier.pkl` using joblib.

Model 2 — Delay Severity Regressor

- Train a regression model to predict `delay_days` using the same feature set as Model 1, but trained only on records where `is_delayed` equals 1.
- Evaluate using Mean Absolute Error (MAE). Print the MAE.
- Save the trained model as `models/delay_regressor.pkl` using joblib.

Model 3 — Carrier Anomaly Detector

- Aggregate the dataset by `carrier_name` to compute the following carrier-level features: average delay days, delay rate as a percentage, average weight, average volume, and total shipment count.
- Run Isolation Forest on these aggregated carrier features to assign each carrier an anomaly score and a binary flag — Normal or Anomalous.
- A carrier is anomalous if the Isolation Forest marks it as an outlier.
- Save the results as a dataframe and export it to `models/carrier_anomalies.csv`.

Model 4 — Route Risk Scorer

- For each unique `origin_country` and `destination_country` pair in the dataset, compute the following: historical delay rate as a percentage, average delay days, customs hold rate as a percentage (percentage of shipments where `customs_cleared` is false), and loss rate as a percentage (percentage of shipments with status Lost).
- Combine these four metrics into a single Route Risk Score between 0 and 100 using a weighted formula of your choice.
- Document the weights you chose and explain why in a comment. Save the results to `models/route_risk_scores.csv`.

- After all four models and outputs are generated, pipeline.py must save a combined output file named pipeline_output.json containing one entry per shipment with the following fields: shipment_id, is_delayed prediction, delay probability, estimated delay days (from Model 2 if predicted delayed, else 0), carrier anomaly flag for that shipment's carrier, and route risk score for that shipment's route.

Task 3 — Set Up the Database

- Choose either MongoDB or MySQL as your database. On backend startup, if the database is empty, automatically load pipeline_output.json into it. Do not require any manual import step.
- All GET endpoints must read from the database. The POST prediction endpoint must write new results to the database before returning them.
- Write a short comment in your database initialization code explaining why you chose MongoDB or MySQL for this use case.

Task 4 — Build the Backend REST API

Build a REST API using FastAPI (Python) or Node.js with Express. If you use Node.js, call the Python models as subprocesses for the prediction endpoint. Load both .pkl model files into memory on server startup — do not reload them on every request.

Implement the following endpoints exactly as specified:

POST /shipments/predict Accepts a JSON body containing a new shipment's pre-departure fields: origin_country, destination_country, carrier_name, shipment_mode, cargo_category, weight_kg, volume_cbm, declared_value_usd, customs_cleared. Loads the delay classifier and delay regressor from memory, runs inference, and returns: delay_prediction (Delayed or On Time), delay_probability (float between 0 and 1), estimated_delay_days (integer, 0 if not delayed), and risk_level (Low if probability below 0.4, Medium if between 0.4 and 0.7, High if above 0.7). Saves the result to the database with a generated shipment_id. Returns 400 with a descriptive message if any required field is missing or of the wrong type.

GET /shipments Returns a paginated list of shipments from the database. Default page size is 20. Supports three optional query parameters: status (filter by shipment status), carrier (filter by carrier_name), and risk_level (Low, Medium, or High — mapped to delay_probability ranges). Each record in the response must include: shipment_id, origin_country, destination_country, carrier_name, status, delay_prediction, delay_probability, and route_risk_score.

GET /shipments/:id Returns the full record for a single shipment by shipment_id. Returns 404 with a message if the id does not exist.

GET /carriers/anomalies Returns all carriers flagged as Anomalous from models/carrier_anomalies.csv loaded into the database. Each entry must include: carrier_name, anomaly_flag, delay_rate_percent, average_delay_days, and total_shipments.

GET /routes/risk Returns all route pairs sorted by route_risk_score in descending order. Each entry must include: origin_country, destination_country, route_risk_score, historical_delay_rate_percent, average_delay_days, and customs_hold_rate_percent.

GET /dashboard/summary Returns aggregate statistics: total_shipments, total_delayed, total_in_transit, total_delivered, average_delay_days, anomalous_carrier_count, and top_5_risk_routes (array of the five highest scoring routes with their scores).

All endpoints must return proper HTTP status codes. All responses must be JSON. Log every incoming request method, path, and response status to the console.

Task 5 — Build the Frontend Dashboard

Build a React.js web application using functional components and hooks. All data must be fetched live from your backend API using HTTP calls.

No hardcoded data, no static JSON imports, no mock responses anywhere in the frontend.

Show a loading spinner on every API call. Show a descriptive error message if any API call fails — do not silently fail or show a blank screen.

Implement the following four pages:

Page 1 — Operations Overview Fetches data from GET /dashboard/summary and displays: four KPI cards showing total shipments, total delayed, total in transit, and average delay days; a bar chart showing shipment count by status; and a horizontal bar chart showing the top 5 highest risk routes with their risk scores. The KPI card for total delayed must display the count and the percentage of total shipments it represents.

Page 2 — Shipment Monitor Fetches data from GET /shipments and displays a table with columns for shipment_id, origin, destination, carrier, status, delay probability as a percentage, and route risk score. Route risk score must appear as a colored badge: green for Low, amber for Medium, red for High. Include three dropdown filters above the table for status, carrier, and risk level — each filter must call the API with the appropriate query parameter rather than filtering the existing data client-side. Clicking any row must open a modal showing the full shipment detail fetched from GET /shipments/:id.

Page 3 — Predict a Shipment A form with input fields for all required prediction fields. On submission, calls POST /shipments/predict and displays the result below the form: delay prediction label, delay probability as a percentage shown in a progress bar, estimated delay days if applicable, and a risk level badge (green for Low, amber for Medium, red for High). If the API returns a 400 error, display the error message from the response — do not show a generic error.

Page 4 — Carrier and Route Intelligence Two sections on the same page. The first section fetches from GET /carriers/anomalies and displays a table of all carriers showing carrier name, anomaly flag, delay rate, and average delay days. Rows where anomaly_flag is Anomalous must have a red background. The second section fetches from GET /routes/risk and displays a sortable table of all routes showing origin,

destination, risk score, delay rate, and customs hold rate. The table must be sortable by risk score in ascending and descending order by clicking the column header.

Task 6 — Containerize the Application

Provide a `docker-compose.yml` at the root of the repository that starts the backend, frontend, and database as three separate services.

Requirements:

- The frontend must be accessible at `http://localhost:3000`
- The backend must be accessible at `http://localhost:8000` (FastAPI) or `http://localhost:5000` (Node.js)
- The database must initialize automatically on first run — no manual import or setup commands
- The ML model `.pkl` files must be available inside the backend container at runtime — they must be copied in during the Docker build, not mounted from the host
- Include a `.env.example` file listing every required environment variable with a placeholder value. The actual `.env` file must not be committed to the repository
- Running `docker-compose up` from the root of the repository after setting up the `.env` file must be the only step required to start the entire application

Task 7 — Test the Full Flow End to End

Before submitting, verify the following manually and confirm each point in your README:

- `docker-compose up` starts all three services with no errors
- `pipeline.py` runs independently and generates all four model files and `pipeline_output.json`

- All six API endpoints return correct responses when called via Postman or curl
- POST /shipments/predict returns a valid prediction for a sample input
- GET /carriers/anomalies returns at least one anomalous carrier
- The frontend dashboard loads data on all four pages without errors
- Filtering on the Shipment Monitor page calls the API with query parameters correctly
- The prediction form displays results correctly after submission

DELIVERABLES

Submit all of the following. Missing any one item will result in an incomplete evaluation.

1. **GitHub Repository Link** — Public repository with the following folder structure: /frontend, /backend, /ml_pipeline, /models, docker-compose.yml, .env.example, README.md
2. **README.md** — Must include: project overview and problem being solved, folder structure explanation, step-by-step instructions to run with Docker, step-by-step instructions to run each service individually without Docker, documentation for every API endpoint with a sample request body and sample response, ML model performance metrics (F1 score for the classifier and MAE for the regressor stated explicitly), dataset description and the distributions applied, and at least 4 screenshots covering each page of the dashboard
3. **shipment_data.csv** — The generated synthetic dataset with a minimum of 500 records and all required fields
4. **pipeline_output.json** — The processed output file containing results for all records in the dataset

5. **models/ directory** — Must contain `delay_classifier.pkl`, `delay_regressor.pkl`, `carrier_anomalies.csv`, and `route_risk_scores.csv`
6. **Working Docker Setup** — `docker-compose up` must start the full application successfully after `.env` setup with no manual steps

SUBMISSION INSTRUCTIONS

Share your GitHub repository link along with a written note of 5 to 8 sentences covering: the algorithms you chose for each model and why, how you designed the synthetic dataset distributions, any trade-offs you made due to time constraints, and what you would improve given more time. Also state your model's F1 score and MAE explicitly in the note.

